

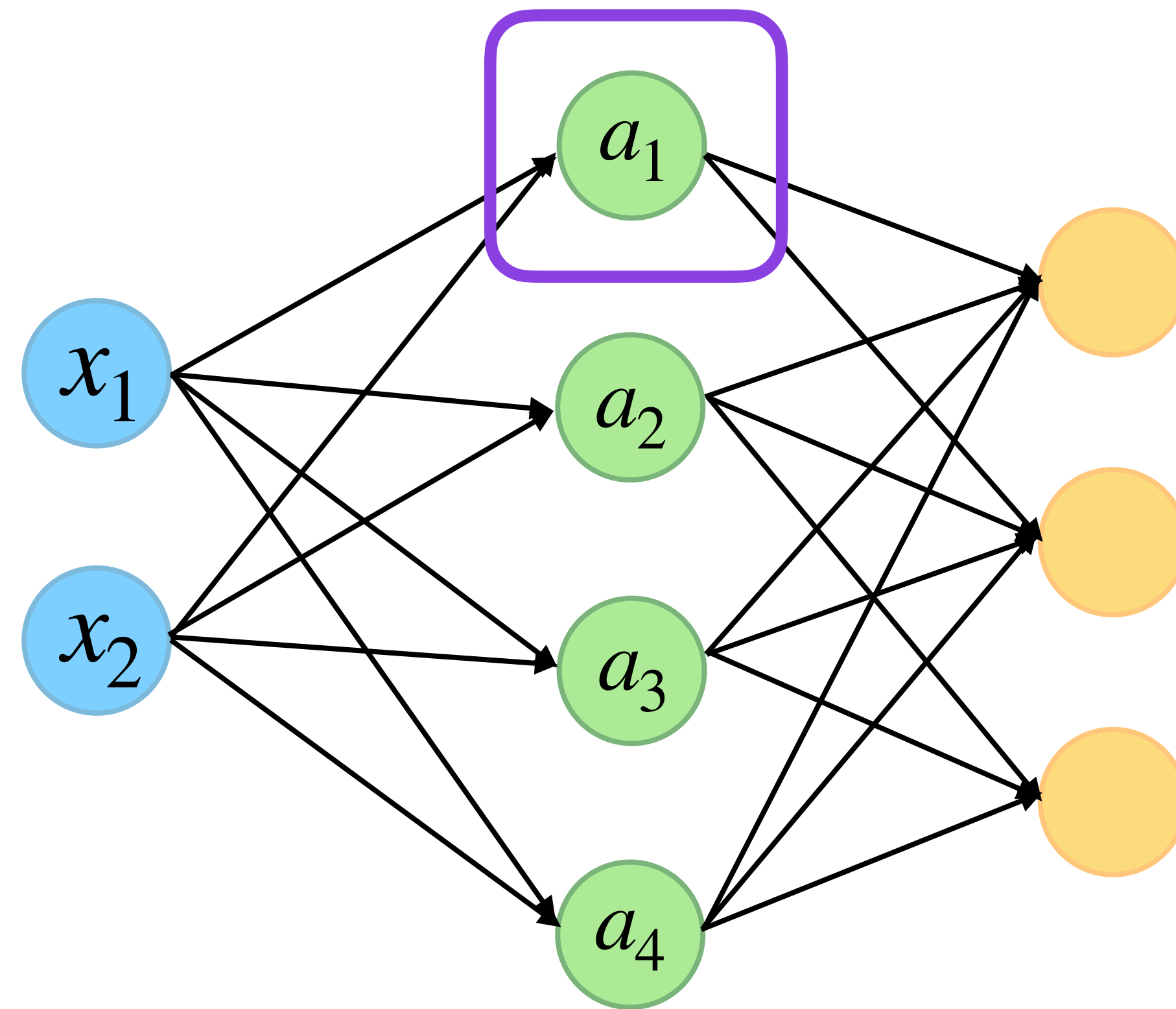
6.6

Improving Convergence with Batch Normalization

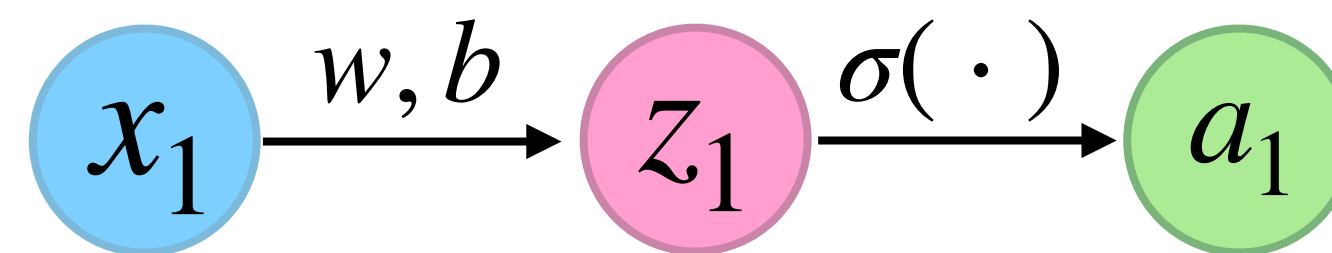
Part 2: Using BatchNorm in PyTorch

Sebastian Raschka and the Lightning AI Team

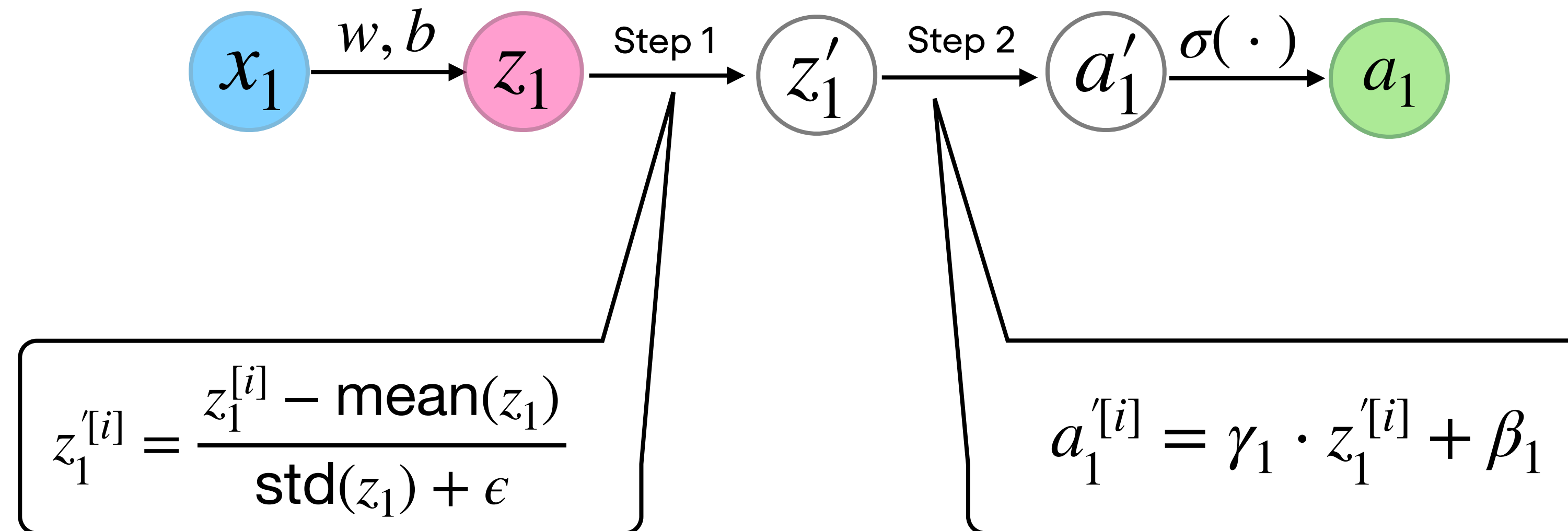
Regular hidden layer activation without BatchNorm



Regular hidden layer activation without BatchNorm



Regular hidden layer activation with BatchNorm



BatchNorm in PyTorch

```
class PyTorchMLP(torch.nn.Module):
    def __init__(self, num_features, num_classes):
        super().__init__()

        self.all_layers = torch.nn.Sequential(
            # 1st hidden layer
            torch.nn.Linear(num_features, 100),
            → torch.nn.BatchNorm1d(100),
            torch.nn.ReLU(),

            # 2nd hidden layer
            torch.nn.Linear(100, 50),
            → torch.nn.BatchNorm1d(50),
            torch.nn.ReLU(),

            # output layer
            torch.nn.Linear(50, num_classes),
        )

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        logits = self.all_layers(x)
        return logits
```

BatchNorm in PyTorch

100 because needs
100 γ and 100 β values

```
class PyTorchMLP(torch.nn.Module):
    def __init__(self, num_features, num_classes):
        super().__init__()

        self.all_layers = torch.nn.Sequential(
            # 1st hidden layer
            torch.nn.Linear(num_features, 100),
            → torch.nn.BatchNorm1d(100),
            torch.nn.ReLU(),

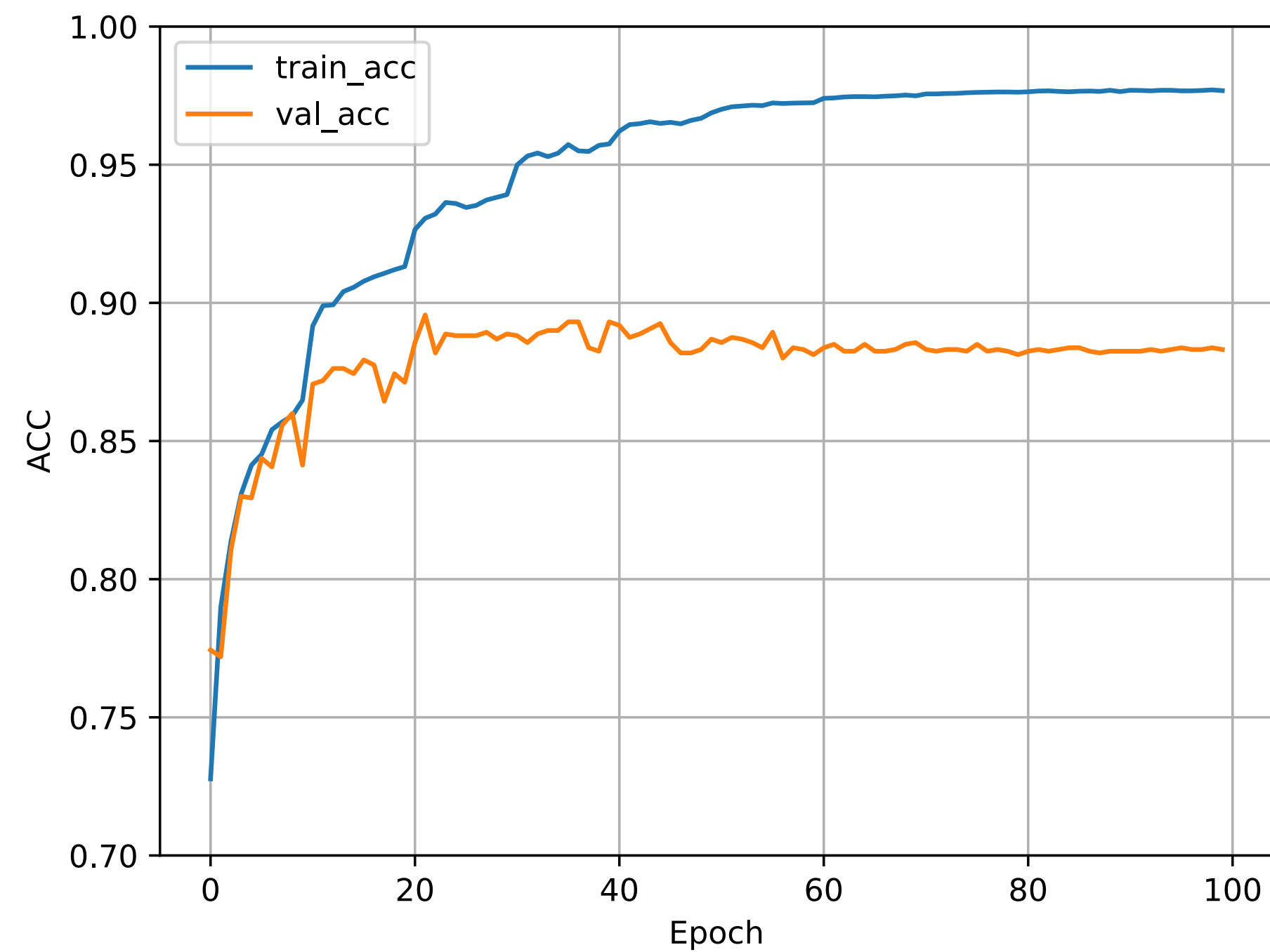
            # 2nd hidden layer
            torch.nn.Linear(100, 50),
            → torch.nn.BatchNorm1d(50),
            torch.nn.ReLU(),

            # output layer
            torch.nn.Linear(50, num_classes),
        )

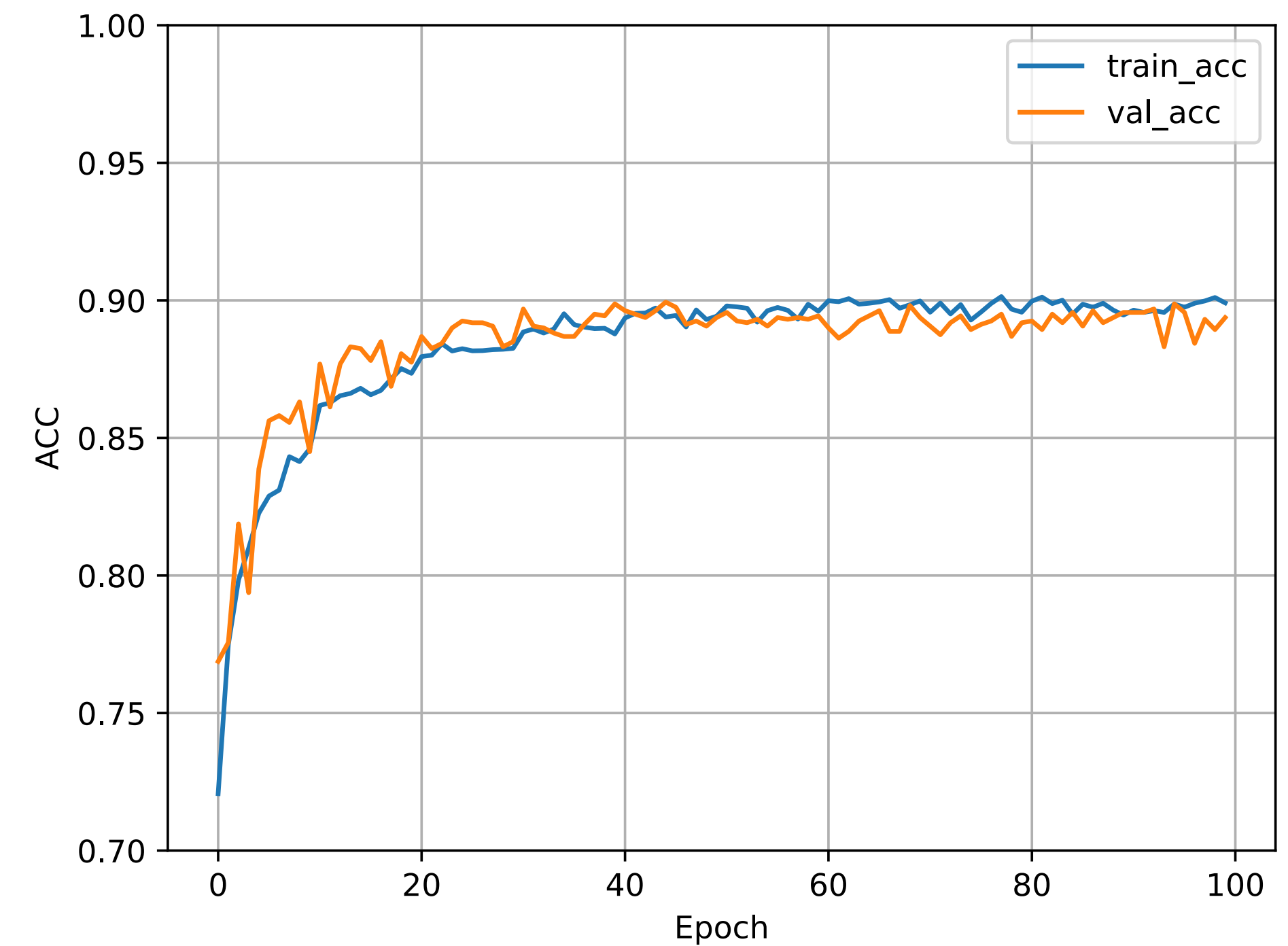
    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        logits = self.all_layers(x)
        return logits
```

Batch normalization (BatchNorm)
can improve convergence and
reduce overfitting

Without BatchNorm



With BatchNorm



BatchNorm during inference

BatchNorm during inference

- Use exponentially weighted average (moving average) of mean and variance [default]
- Alternatively, one also use global training set mean and variance

BatchNorm during inference

- Use exponentially weighted average (moving average) of mean and variance [default] → `model.eval()`
- Alternatively, one also use global training set mean and variance

Why does BatchNorm work?
Hot topic of discussion!
(see pointers below)

NEXT: Reducing overfitting with Dropout